

Graph-to-Tree: Topological Decomposition for Self-Supervised Learning

Yejiang Wang
Northeastern University
Xiamen University
Shenyang & Xiamen, China
wyejiang@gmail.com

Yuhai Zhao*
Northeastern University
Shenyang, China
zhaoyuhai@mail.neu.edu.cn

Fangting Li
Northeastern University
Shenyang, China
neulifangting@163.com

Jiapu Wang
Nanjing University of Science and
Technology
Nanjing, China
jiapuwang9@gmail.com

Meixia Wang
Northeastern University
Shenyang, China
wangmx7@mails.neu.edu.cn

Ling Li
Shanxi University
Taiyuan, China
lilingsxu@sxu.edu.cn

Miaomiao Huang
Northeastern University
Shenyang, China
huangmiaomiao@stumail.neu.edu.cn

Zhengkui Wang
Singapore Institute of Technology
Singapore, Singapore
zhengkui.wang@singaporetech.edu.sg

Shirui Pan
Griffith University
Gold Coast, Australia
s.pan@griffith.edu.au

Abstract

Every graph hides a tree: through tree decomposition—a foundational tool in modern graph theory with broad applications such as in computational power networks, any network can be unfolded into a hierarchy of overlapping vertex bags whose backbone is a tree. Leveraging this powerful lens, we propose Topological Decomposition for Self-supervised Learning (TopDSL), a framework that injects multi-scale signals into graph representation learning. Concretely, we: 1) decompose the input graph into tree structures with bags representing local structural contexts; 2) compute bag-level roles via closeness centrality for nodes and local edge betweenness for edges, and aggregate these scores across bags to capture context-dependent importance (e.g., local structural bridges); 3) convert the resulting importance and attribute-stability scores into a context-aware augmentation policy that adaptively perturbs nodes, edges, and features—preserving local bridges, honoring multi-community vertices, and attenuating noisy global hubs; 4) construct a new structural similarity loss for contrastive learning, which fuses traditional graph-based proximity with a novel tree-based similarity derived from node co-occurrence in decomposition bags; 5) demonstrate that our framework achieves superior performance over state-of-the-art baselines on various graph learning benchmarks.

CCS Concepts

• Theory of computation → Graph algorithms analysis; • Information systems → Data mining.

*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792286>

Keywords

Graph Neural Network; Self-supervised Learning; Graph Learning

ACM Reference Format:

Yejiang Wang, Yuhai Zhao, Fangting Li, Jiapu Wang, Meixia Wang, Ling Li, Miaomiao Huang, Zhengkui Wang, and Shirui Pan. 2026. Graph-to-Tree: Topological Decomposition for Self-Supervised Learning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792286>

1 Introduction

Tree decomposition [36] is a fundamental concept in graph theory that provides a canonical way to reveal the hierarchical organization of graphs. By partitioning a graph into overlapping cliques (called “bags”) and connecting these bags in a tree structure, tree decomposition exposes the underlying skeletal structure that governs complex networks [8]. As illustrated in Fig.1, a graph $\mathcal{G}$ can be decomposed into a tree $\mathcal{T}$ where each node represents a bag containing a subset of vertices from $\mathcal{G}$. This decomposition satisfies three key properties: 1) every edge in $\mathcal{G}$ is contained within at least one bag, 2) for any vertex v , the bags containing v form a connected subtree, and 3) any two bags that share a common vertex are connected within the tree. For example, a bag like $\{5, \dots, 10\}$ might represent a local community, while an adjacent bag $\{10, 11\}$ connects to it by sharing vertex 10. In this way, the decomposition highlights local clusters and their intersections within $\mathcal{G}$ —features that are often obscured in the graph’s raw topology.

Tree decomposition has found widespread applications across diverse scientific domains, serving as a powerful analytical tool in computing power networks, computational biology [60], social network analysis [26], and computational complexity theory [12]. For instance, in protein interaction networks, tree decomposition naturally captures functional modules and their hierarchical relationships [57], while in social networks, it reveals overlapping

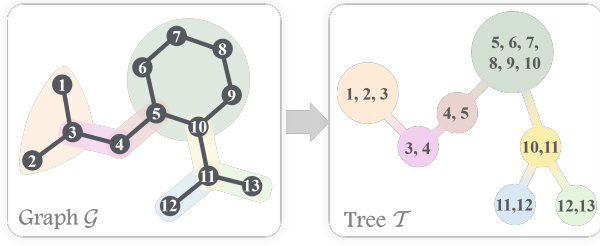


Figure 1: An example of tree decomposition. A graph $\mathcal{G}$ is decomposed into overlapping vertex bags connected in a tree $\mathcal{T}$. Each bag contains a subset of vertices, and every edge in $\mathcal{G}$ is covered by at least one bag.

community structures and their interconnections [1]. Given this rich structural information embedded in tree decompositions, we would naturally expect graph learning models to leverage these hierarchical patterns and multi-scale contexts to produce more informative and structurally-aware node embeddings that reflect both local community memberships and global positional roles.

Despite rapid progress, current approaches in graph representation learning—particularly within the widely studied paradigm of self-supervised graph learning [54], such as graph contrastive learning (which typically augments the original graph, e.g., by randomly removing edges, to generate two contrasting views, and then applies contrastive constraints to maximize similarity between representations of the same node (positive pairs) across these views while minimizing similarity for different nodes (negative pairs))—still primarily address graph structure from a *global* perspective. For instance, GRAPHCL [63] applies uniform random perturbations to the entire graph, while GCA [68] relies on metrics such as degree or PageRank;

Ad-GCL [45] and AutoGCL [61] learn edge masks with graph-level adversaries or controllers; and Cr-GCL [46] enforces invariance constraints within non-overlapping, globally identified communities. These strategies overlook the rich insights that tree decomposition offers into the graph itself, such as: 1) *Local structural bridges*—edges that appear insignificant globally but serve as the sole connectors between dense subcomponents within specific local contexts (bags). While such edges may have low global centrality, removing them can cause severe topological information loss within their local environments [33, 48]; 2) *Multi-role vertices*—graphs often contain nodes that simultaneously play distinct roles across overlapping communities [15, 17], but global or disjoint partitions collapse these nuanced functions; 3) *Robustness*—spurious super-hubs from data errors can distort global centrality or role identification [53], whereas tree decomposition localizes computations to bags, confining a noisy hub’s influence to few contexts and making the process more robust.

To bridge this gap, we introduce Graph-to-Tree: Topological Decomposition for Self-supervised Learning (TopDSL), a tree-aware paradigm that treats the decomposition itself as both a structural *prior* and a computational *scaffold*. In summary, our main contributions are: 1) We decompose input graphs into tree structures where bags capture overlapping local structural contexts, providing a natural hierarchy from local neighborhoods to global topology; 2) We

compute context-aware importance scores through bag-localized metrics—closeness centrality for nodes and edge betweenness for edges within each bag—then aggregate across bags using bag-level importance weights, enabling identification of elements critical in specific contexts (e.g., local bridges) that global metrics overlook; 3) We develop an adaptive augmentation policy that converts structural importance and attribute stability scores into perturbation probabilities, preserving semantically coherent neighborhoods while allowing meaningful variations; 4) We introduce a novel structural similarity measure for contrastive objective that fuses graph proximity (via personalized PageRank) with tree-based similarity derived from node co-occurrence patterns in decomposition bags, capturing both flat topological and hierarchical structural relationships; 5) Extensive experiments on benchmark datasets demonstrate that TopDSL outperforms SOTA baselines.

2 Related Work

Tree Decomposition. Tree decomposition, introduced by Robertson and Seymour [36], is a fundamental tool in graph theory that systematically partitions complex networks into overlapping substructures organized in a tree-like hierarchy [26, 32]. Tree decomposition has found widespread applications across scientific domains. In computational biology, it has been applied to protein structure prediction [57] and PPI network querying through tools like QNet [14] and feedback vertex set approaches [6]. The Infrared framework [60] leverages tree decomposition for RNA design and phylogenetic analysis, while social network analysis has benefited from its ability to reveal community hierarchies [1]. In graph generation, TD-GEN [43] demonstrates how tree decomposition can guide the synthesis of realistic graph structures by hierarchically constructing graphs from their tree-structured backbones. Empirical studies demonstrate that tree decomposition naturally captures real-world network properties. Bodlaender et al. [26] showed that many practical graphs exhibit small treewidth, efficiently decomposing into manageable components. This aligns with observations that real networks often display dense cores connected to tree-like peripheries [31], making tree decomposition well-suited for capturing both local clustering and global connectivity patterns. Despite its success in other domains, few works exploit tree decomposition in graph learning. To the best of our knowledge, our work is the first to bridge this gap, introducing tree decomposition as a principled framework for self-supervised graph learning.

Self-Supervised Learning on Graphs. Self-supervised learning (SSL) has emerged as a powerful paradigm to learn expressive representations from vast amounts of unlabeled data, thereby mitigating the dependency on costly manual annotations [11, 13, 18, 38]. Within the graph domain, Graph contrastive learning (GCL) has become the most prominent and extensively researched approach [21, 25, 51, 66]. GCL has demonstrated significant success across a multitude of applications, such as social networks, biological networks [16, 39, 50, 65]. The central tenet of modern GCL algorithms is to maximise the agreement between two augmented “views” of the same graph or node. Consequently, the design of the graph-augmentation strategy is critical, as its primary role is to generate semantically invariant views that serve as positive pairs for the contrastive objective [63, 67, 68].

These augmentation strategies are broadly either rule-based or learning-based. Rule-based methods apply pre-defined stochastic perturbations, such as random node/edge dropping [37, 56, 63]. Learning-based approaches generate views in a data-driven manner—for instance, through adversarial training like in AD-GCL [45] or by learning to identify latent structures such as communities [46]. However, these strategies often apply perturbations globally or learn implicit structures, thereby overlooking the explicit, hierarchical knowledge that can be captured by graph decompositions.

3 Backgrounds

Problem Statement. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ denote an attributed graph, where $\mathcal{V} = \{v_1, \dots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. The graph's structure is represented by an adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$, where $A_{ij} = 1$ if $(v_i, v_j) \in \mathcal{E}$ and 0 otherwise. For any subset $S \subseteq \mathcal{V}$, the induced subgraph $\mathcal{G}[S] = (S, E_S)$ contains all edges $(u, v) \in \mathcal{E}$ where both $u, v \in S$. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^d$, and the collection of all node features forms the matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$, where d is the feature dimension. The goal of our work is to learn a mapping function, or encoder, $f_\theta : \mathcal{G} \rightarrow \mathbb{R}^{n \times q}$ that generates a low-dimensional representation vector (i.e., an embedding) $\mathbf{z}_v \in \mathbb{R}^q$ for each node $v \in \mathcal{V}$, where $q \ll n, d$. This process is entirely self-supervised, meaning it does not rely on any node labels.

Tree Decomposition. For a graph $\mathcal{G}$, a *tree decomposition* is a pair $(\mathcal{T}, \mathcal{B})$, where $\mathcal{T} = (I, F)$ is a junction tree with node set I and edge set F , and $\mathcal{B} = \{B_i \mid i \in I\}$ is a collection of subsets of $\mathcal{V}$ called *bags*, such that the following properties are satisfied: 1) *Node Coverage*: $\bigcup_{i \in I} B_i = \mathcal{V}$ (every node in $\mathcal{G}$ appears in at least one bag). 2) *Edge Coverage*: For every edge $(u, v) \in \mathcal{E}$, there exists some $i \in I$ such that $\{u, v\} \subseteq B_i$ (every edge is contained within at least one bag). 3) *Coherence Property*: For every node $v \in \mathcal{V}$, the subgraph of $\mathcal{T}$ induced by tree nodes $\{i \in I \mid v \in B_i\}$ is connected (bags containing the same node form a connected subtree).

Treewidth. The *width* of a tree decomposition $(\mathcal{T}, \mathcal{B})$ is defined as $w(\mathcal{T}, \mathcal{B}) = \max_{i \in I} |B_i| - 1$. The *treewidth* of a graph $\mathcal{G}$, denoted as $\text{tw}(\mathcal{G})$, is the minimum width over all possible tree decompositions of $\mathcal{G}$:

$$\text{tw}(\mathcal{G}) = \min_{(\mathcal{T}, \mathcal{B}) \in \mathcal{D}(\mathcal{G})} w(\mathcal{T}, \mathcal{B}) \quad (1)$$

where $\mathcal{D}(\mathcal{G})$ denotes the set of all valid tree decompositions.

While finding the optimal tree decomposition with minimum treewidth is NP-hard in general [12], various approximation algorithms exist that can compute tree decompositions with reasonably small width for practical graphs [8]. For our framework, we use the notation $(\mathcal{T}, \mathcal{B}) = \text{TreeDecomp}(\mathcal{G})$ to denote a generic tree decomposition algorithm that takes a graph $\mathcal{G}$ as input and returns a tree decomposition $(\mathcal{T}, \mathcal{B})$.

Induced Subgraphs for Tree. For each bag B_i in the tree decomposition, the *induced subgraph* $\mathcal{G}[B_i] = (B_i, E_i)$, where $E_i = \{(u, v) \in \mathcal{E} \mid u, v \in B_i\}$ contains all edges between nodes within bag B_i . These induced subgraphs serve as local structural contexts that capture dense connectivity patterns within each bag, forming the basis for our context-aware analysis.

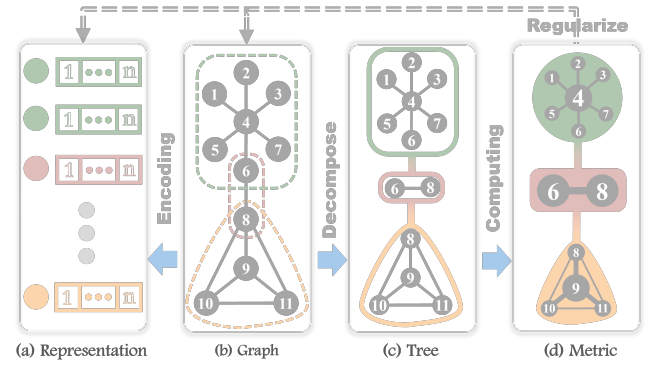


Figure 2: The TopDSL framework pipeline. An input graph (b) is decomposed into a tree of bags (c), which informs the computation of structural metrics (d) used to regularize the model and produce robust representations (a).

4 Methodology

Recent advances in graph contrastive learning (GCL) rely on data augmentation, which implicitly assumes label invariance—that is, the core semantics of a node should remain unchanged after perturbations are applied to the graph structure or features. However, existing augmentation strategies often operate with a global perspective, which often leads to the neglect of context-dependent structures such as local bridges. To address these limitations, the core of our proposed framework, Topological Decomposition for Self-supervised Learning (TopDSL), is to leverage tree decomposition to identify the multi-scale hierarchy inherent in the graph. This allows us to perform an adaptive augmentation guided by fine-grained importance scores computed for nodes and edges within their specific local contexts, thereby preserving vital structural information. Furthermore, our contribution extends beyond augmentation; we also harness the decomposition to redefine structural similarity for the objective itself. By constructing a novel similarity metric from node co-occurrence patterns in the decomposition bags, we ensure that the learned representations reflect not just graph proximity but also hierarchical relatedness, forcing nodes in similar structural contexts to have more similar embeddings.

As illustrated in Fig.2, our framework proceeds in four main stages. First, we apply a tree decomposition algorithm to the input graph to obtain a tree-structured hierarchy of overlapping bags. Second, we compute context-aware importance scores—using closeness centrality for nodes and edge betweenness for edges within each bag—and aggregate these scores to derive adaptive perturbation probabilities for generating two distinct contrastive views. Third, a graph encoder, such as a graph neural network, processes these two views to produce node embeddings. Finally, our novel structural decomposition-aware contrastive loss function constrains these embeddings, maximizing agreement between nodes based on our tree-informed similarity scores.

4.1 Graph to Tree

We begin our framework by applying a tree decomposition algorithm to partition the input graph $\mathcal{G}$ into a hierarchical structure.

Tree decomposition algorithms in graph theory are broadly categorized into two types: exact algorithms that guarantee optimal treewidth, and heuristic (approximation) algorithms that provide near-optimal solutions [3]. Since determining the treewidth of an arbitrary graph is NP-complete [5], exact computation becomes intractable for large real-world networks. To overcome this computational barrier, we employ heuristic algorithms that seek to find a *sufficiently good* tree decomposition $(\mathcal{T}, \mathcal{B})$ in polynomial time, typically near-linear complexity. For example, we can utilize the minimum degree heuristic [7], which iteratively selects vertices with the smallest degree for elimination.

The rationale for employing heuristic algorithms extends beyond computational efficiency to align with our core objective: leveraging the resulting bags as meaningful local contexts for calculating relative importance rather than minimizing treewidth. Heuristics like minimum degree implicitly seek sparse separators while preserving dense substructures during node elimination [9], naturally yielding bags that correspond to intrinsic communities or tightly-knit regions [1]. This alignment ensures that even non-optimal decompositions effectively partition graphs into structurally relevant local environments. Our approach's robustness further stems from computing relative centrality within these bags—consider an edge $e=(u, v)$ bridging communities C_1 and C_2 . Any reasonable decomposition must create *bridging bags* containing both endpoints along with members from both communities, where e 's betweenness centrality remains high as a critical connector. While heuristic decompositions might produce larger bags or distribute structures differently, they preserve the qualitative distinction between *critical* and *non-critical* elements. Since our model learns from relative importance differences rather than absolute centrality values, heuristic algorithms represent an optimal trade-off between computational feasibility and theoretical effectiveness, prioritizing context quality over treewidth optimality.

4.2 Tree for Augmentation

Building upon the tree decomposition $(\mathcal{T}, \mathcal{B})$ obtained in the previous stage, we now compute context-aware importance scores for both nodes and edges within their respective local structural contexts. These scores serve as the foundation for our adaptive augmentation strategy, which generates semantically meaningful contrastive views by selectively perturbing graph elements based on their contextual significance. Our approach operates on two complementary levels: first computing bag-level importance weights that reflect the structural significance of each local context within the global hierarchy, then aggregating localized centrality measures across bags that preserve critical structures.

Node-centric Augmentation. Our node augmentation strategy encompasses both structural (node dropping) and feature perturbation (feature masking), guided by context-aware importance scores derived from the tree decomposition. We begin by computing bag-level importance scores using *closeness centrality* within the tree structure $\mathcal{T}$. For each bag B_i , its structural importance is defined as:

$$S_b(i) = \frac{1}{\sum_{j \in I, j \neq i} d_{\mathcal{T}}(i, j)} \quad (2)$$

where $d_{\mathcal{T}}(i, j)$ represents the shortest path distance between bags i and j in the tree $\mathcal{T}$, and I denotes the set of all tree nodes. Next, we compute node-level structural importance by evaluating closeness centrality within each induced subgraph $\mathcal{G}[B_i]$. For node $v_k \in B_i$, its local centrality is:

$$S_n^i(k) = \frac{1}{\sum_{v_l \in B_i, l \neq k} d_{\mathcal{G}[B_i]}(v_k, v_l)} \quad (3)$$

where $d_{\mathcal{G}[B_i]}(v_k, v_l)$ denotes the shortest path distance between nodes v_k and v_l within the induced subgraph. Since nodes may appear in multiple bags, we aggregate their importance scores using a *weighted maximum* strategy: $S_{nb}(k) = \max_{i: v_k \in B_i} \{S_n^i(k) \times S_b(i)\}$. The node dropping probability is then computed as

$$p_{nb}(k) = p_0 \times (1 - \tilde{S}_{nb}(k)) \quad (4)$$

where $\tilde{S}_{nb}(k)$ represents the normalized structural importance and p_0 is a hyperparameter controlling the base dropping rate.

For feature perturbation, we assess local attribute stability by measuring the homophily within each bag. For node $v_j \in B_i$, we compute its attribute consistency with local neighbors as:

$$S_a^i(k) = \frac{1}{|\mathcal{N}(v_k) \cap B_i|} \sum_{v_l \in \mathcal{N}(v_k) \cap B_i} \cos(\mathbf{x}_k, \mathbf{x}_l) \quad (5)$$

where $\mathcal{N}(v_k)$ represents the neighborhood of node v_k in the original graph $\mathcal{G}$, and $\cos(\cdot, \cdot)$ computes cosine similarity between feature vectors. The final attribute stability score is aggregated as $S_{ab}(k) = \max_{i: v_k \in B_i} \{S_a^i(k) \times S_b(i)\}$, leading to feature perturbation probability

$$p_{ab}(k) = p_0 \times (1 - \tilde{S}_{ab}(k)). \quad (6)$$

where $\tilde{S}_{ab}(k)$ represents the normalized importance. Feature masking is performed by sampling a masking matrix $\mathbf{M} \in \{0, 1\}^{n \times d}$ where $\mathbf{M}_{kl} \sim \text{Bernoulli}(p_{ab}(k))$, yielding the augmented feature matrix $\tilde{\mathbf{X}} = \mathbf{X} \odot (\mathbf{1} - \mathbf{M}) + \mathbf{N} \odot \mathbf{M}$, with $\mathbf{N} \sim \mathcal{N}(\mathbf{X}, \Sigma)$ providing Gaussian noise.

Edge-centric Augmentation. Our edge augmentation focuses on preserving local structural bridges that serve as critical connectors within specific contexts while allowing perturbation of less significant edges. We employ local edge betweenness centrality as our primary metric for edge importance within individual bags. For edge $e_{uv} = (u, v)$ within bag B_i , its local betweenness centrality is computed as:

$$S_e^i(e_{uv}) = \sum_{s, t \in B_i, s \neq t} \frac{\sigma_{st}(e_{uv})}{\sigma_{st}} \quad (7)$$

where σ_{st} denotes the total number of shortest paths between nodes s and t in the induced subgraph $\mathcal{G}[B_i]$, and $\sigma_{st}(e_{uv})$ represents the number of those paths that traverse edge e_{uv} . This metric effectively identifies local bridges that may appear insignificant globally but serve as crucial connectors within their local contexts. Since edges may span multiple bags according to the edge coverage property of tree decomposition, we aggregate their importance scores across all containing bags. For edge e_{uv} , let $\mathcal{B}(e_{uv}) = \{i \in I : u, v \in B_i\}$ denote the set of bags containing both endpoints. The final edge importance is computed using a weighted aggregation strategy:

$$S_{eb}(e_{uv}) = \max_{i \in \mathcal{B}(e_{uv})} \{S_e^i(e_{uv}) \times S_b(i)\} \quad (8)$$

The edge dropping probability is inversely related to importance:

$$p_{eb}(e_{uv}) = p_0 \times (1 - \tilde{S}_{eb}(e_{uv})) \quad (9)$$

where $\tilde{S}_{eb}$ represents the normalized edge importance scores. Edge augmentation is performed by sampling a binary masking matrix $\mathbf{E} \in \{0, 1\}^{n \times n}$ where $E_{uv} \sim \text{Bernoulli}(p_{eb}(e_{uv}))$, producing the augmented adjacency matrix $\tilde{\mathbf{A}} = \mathbf{A} \odot (\mathbf{1} - \mathbf{E})$.

4.3 Tree for Loss

After generating the two augmented views $\tilde{\mathcal{G}}_1$ and $\tilde{\mathcal{G}}_2$ through our tree-guided adaptive augmentation, we employ a graph encoder f_θ (e.g., graph neural network) to produce node embeddings $\mathbf{Z}^i = f_\theta(\tilde{\mathcal{G}}_i) \in \mathbb{R}^{n \times q}$. To effectively train these representations in a self-supervised manner, we introduce a novel topological decomposition-aware contrastive loss that incorporates tree-informed node similarities. This loss extends traditional contrastive objectives by weighting the similarity between node pairs based on their structural affinity, derived from both the original graph topology and the hierarchical contexts revealed by the tree decomposition $(\mathcal{T}, \mathcal{B})$. By doing so, we ensure that embeddings of nodes sharing similar structural roles—whether through direct topological proximity or shared bag memberships—are pulled closer, while dissimilar nodes are pushed apart, capturing multi-scale invariances.

We first compute a graph-based similarity matrix $S_g \in [0, 1]^{n \times n}$ using Personalized PageRank (PPR) [4], which measures the proximity of node v_t relative to source node v_s via random walks with restart. Formally, the PPR score $\pi(s, t)$ is defined as the stationary distribution of a random walk starting at s with restart probability α :

$$\pi(s, t) = (1 - \alpha) \sum_{u \in \mathcal{N}(t)} \frac{\pi(s, u)}{|\mathcal{N}(u)|} + \alpha \cdot \mathbb{I}(s = t), \quad (10)$$

where $\mathbb{I}$ is the indicator function, and $\mathcal{N}(u)$ denotes the neighbors of u in $\mathcal{G}$. The stationary scores obtained from the above equation constitute the graph-based similarity score $S_g(s, t)$.

Complementing this, we derive a tree-based similarity matrix $S_t \in [0, 1]^{n \times n}$ from bag co-occurrence, leveraging the intuition that nodes frequently appearing in the same bags share core structural environments, such as overlapping communities or functional modules. Let $B(v) = \{i \in I \mid v \in B_i\}$ denote the set of bags containing node v . We employ a Jaccard-inspired index [29] for pairwise similarity:

$$S_t(u, v) = \frac{|B(u) \cap B(v)|}{|B(u) \cup B(v)|}, \quad (11)$$

which quantifies shared structural contexts across the decomposition hierarchy. This metric naturally encodes multi-scale relationships, as bags represent localized clusters connected through the tree $\mathcal{T}$.

To fuse these complementary views, we first normalize both matrices to $\tilde{S}_g$ and $\tilde{S}_t$. The final similarity matrix is obtained through weighted averaging:

$$S_{gt} = \alpha \cdot \tilde{S}_g + (1 - \alpha) \cdot \tilde{S}_t, \quad (12)$$

where $\alpha \in [0, 1]$ is a tunable hyperparameter. Our objective moves beyond the traditional binary distinction of positive and negative pairs by treating the fused similarity $S_{gt}(u, v)$ as a soft ground-truth label for the correspondence between any two nodes u and v . We align the embedding similarity with this structural similarity using

Algorithm 1 TopDSL

```

1: Input: Graph  $\mathcal{G}$ , encoder  $f_\theta$ ,  $p_0$ ,  $\alpha$ .
2:  $(\mathcal{T}, \mathcal{B}) \leftarrow \text{TreeDecomp}(\mathcal{G})$  // Tree decomp.
3: while no converge do
4:    $\{S_b(i)\} \leftarrow \text{closeness}(\mathcal{T})$  // Bag importance.
5:    $S_{nb}(k) = \max_i S_n^i(k) \times S_b(i)$  // Node imp.
6:    $S_{ab}(k) = \max_i S_a^i(k) \times S_b(i)$  // Node attr. stability.
7:    $S_{eb}(e_{uv}) = \max_i S_e^i(e_{uv}) \times S_b(i)$  // Edge imp.
8:    $p_{nb}(k) = p_0(1 - S_{nb}(k))$  // Node drop prob.
9:    $p_{ab}(k) = p_0(1 - S_{ab}(k))$  // Feature pert. prob.
10:   $p_{eb}(e_{uv}) = p_0(1 - \tilde{S}_{eb}(e_{uv}))$  // Edge drop prob.
11:   $\tilde{\mathcal{G}}_1, \tilde{\mathcal{G}}_2 \leftarrow \text{augment}(\mathcal{G}; p_{nb} \mid p_{ab} \mid p_{eb})$  // Gen. views.
12:   $\mathbf{Z}^1 = f_\theta(\tilde{\mathcal{G}}_1), \mathbf{Z}^2 = f_\theta(\tilde{\mathcal{G}}_2)$  // Embeddings.
13:   $S_g \leftarrow \text{PPR}(\mathcal{G})$  // Graph sim.
14:   $S_t(u, v) = \frac{|B(u) \cap B(v)|}{|B(u) \cup B(v)|}$  // Tree sim.
15:   $S_{gt} = \alpha \cdot \tilde{S}_g + (1 - \alpha) \cdot \tilde{S}_t$  // Fuse.
16:  Compute overall loss  $\mathcal{L}$  using Eq.(13) // Loss.
17:  Update  $f_\theta$  to min  $\mathcal{L}$ .
18: end while
19: return  $f_\theta$ .
```

a generalized binary cross-entropy objective over all node pairs. Let $\Theta_{uv} = (\mathbf{z}_u^1)^T \mathbf{z}_v^2$ be the agreement score between the embeddings of nodes u and v from the two views. The loss is defined as:

$$\mathcal{L} = \sum_{u, v \in \mathcal{V}} \sigma_{sp}(\Theta_{uv} \cdot (1 - S_{gt}(u, v))) + \sigma_{sp}(-\Theta_{uv} \cdot S_{gt}(u, v)) \quad (13)$$

where $\sigma_{sp}(x) = \log(1 + e^x)$ is the softplus function.

In this formulation, the model is trained to predict a high probability $\sigma(\Theta_{uv})$ when the structural similarity $S_{gt}(u, v)$ is high, and a low probability otherwise. This allows the objective to leverage the full spectrum of pairwise relationships, encouraging embeddings to capture nuanced structural roles rather than adhering to a rigid positive/negative dichotomy.

4.4 Complexity Analysis

The overall time complexity of our proposed TopDSL framework is composed of three main parts: tree decomposition, augmentation score calculation, and loss computation. We first address the computational cost of obtaining the graph's hierarchical structure and preparing for augmentation. The tree decomposition is performed using the minimum degree heuristic. While each node elimination step can introduce fill-in edges and form new cliques, the total number of operations is bounded. An amortized analysis reveals that by efficiently managing degree updates, the total cost for this stage is effectively linear, scaling at $O(n + m)$, where $n = |\mathcal{V}|$ and $m = |\mathcal{E}|$. Following decomposition, we compute importance scores for augmentation. This process is highly localized within the r bags of the decomposition. For each bag, which has an average size of w (where $w \ll n$ for graphs with small treewidth), we calculate node and edge centralities. By leveraging optimized algorithms for closeness and betweenness centrality that run on these small induced subgraphs, the total complexity for this step is contained to $O(rw^2)$, making it scalable as it avoids computations over the entire graph.

Table 1: Supervised and unsupervised classification accuracy (%) on TU datasets. The highest unsupervised performance is highlighted in bold, and the second-highest performance is underlined.

Model	NCI1	PROTEINS	DD	MUTAG	COLLAB	RDT-B	RDT-M5K	IMDB-B
GCN	80.20 ± 0.14	74.92 ± 0.33	76.24 ± 0.14	85.63 ± 0.24	79.01 ± 0.18	50.00 ± 0.10	20.00 ± 0.10	70.45 ± 0.37
GIN	82.75 ± 0.19	76.28 ± 0.28	78.91 ± 0.13	89.47 ± 0.16	80.23 ± 0.19	92.44 ± 0.25	56.50 ± 0.24	73.70 ± 0.60
SP	73.53 ± 0.16	75.07 ± 0.54	>1d	85.25 ± 0.24	–	64.13 ± 0.04	39.64 ± 0.02	55.62 ± 0.02
GK	66.06 ± 0.12	71.67 ± 0.55	78.53 ± 0.03	81.71 ± 0.21	71.81 ± 0.31	77.32 ± 0.02	41.06 ± 0.08	65.93 ± 0.10
WL	80.01 ± 0.50	72.92 ± 0.56	<u>79.78 ± 0.36</u>	80.76 ± 0.30	69.30 ± 0.42	68.82 ± 0.41	46.06 ± 0.21	72.30 ± 0.44
DGK	<u>79.98 ± 0.36</u>	73.21 ± 0.61	74.79 ± 0.32	87.51 ± 0.65	64.43 ± 0.48	78.35 ± 0.43	41.49 ± 0.32	67.09 ± 0.37
NODE2VEC	54.93 ± 0.16	57.58 ± 0.36	–	72.62 ± 1.02	56.12 ± 0.02	–	–	50.25 ± 0.09
SUB2VEC	52.82 ± 0.15	53.06 ± 0.56	54.33 ± 0.24	61.17 ± 1.59	55.26 ± 0.15	71.48 ± 0.42	36.68 ± 0.42	55.34 ± 0.15
GRAPH2VEC	73.21 ± 0.18	73.33 ± 0.21	79.32 ± 0.29	83.28 ± 0.93	71.10 ± 0.54	75.78 ± 0.96	46.86 ± 0.26	71.16 ± 0.05
INFOGRAPH	76.33 ± 0.79	74.27 ± 0.43	73.02 ± 1.31	88.65 ± 1.09	70.41 ± 0.34	83.06 ± 0.82	53.45 ± 1.10	72.97 ± 0.49
GRAPHCL	77.54 ± 0.34	74.41 ± 0.35	78.49 ± 0.36	86.62 ± 1.23	71.28 ± 1.16	89.50 ± 0.73	55.85 ± 0.33	71.29 ± 0.38
AD-GCL	74.15 ± 0.62	73.33 ± 0.36	75.74 ± 0.68	88.70 ± 1.66	72.01 ± 0.53	90.04 ± 0.76	54.55 ± 0.40	70.35 ± 0.59
JOAOV2	78.36 ± 0.21	74.02 ± 0.53	77.36 ± 0.29	87.23 ± 1.12	69.05 ± 0.22	86.36 ± 0.30	55.77 ± 0.23	70.12 ± 0.25
RGCL	78.14 ± 0.89	75.11 ± 0.58	78.77 ± 0.52	87.62 ± 1.16	70.85 ± 0.63	<u>90.27 ± 0.54</u>	<u>56.39 ± 0.37</u>	71.67 ± 0.76
SIMGRACE	79.06 ± 0.15	75.26 ± 0.24	77.13 ± 0.88	89.23 ± 1.25	71.26 ± 0.51	89.41 ± 0.33	55.91 ± 0.23	71.22 ± 0.30
SEGA	79.15 ± 0.52	<u>76.00 ± 0.68</u>	78.81 ± 0.33	<u>90.23 ± 0.89</u>	<u>74.10 ± 0.62</u>	90.12 ± 0.46	56.08 ± 0.40	<u>73.60 ± 0.33</u>
AUTOGCL	78.39 ± 0.58	69.80 ± 0.38	75.76 ± 0.93	85.22 ± 1.43	71.40 ± 0.26	86.56 ± 1.31	55.68 ± 0.23	72.26 ± 0.34
TopDSL	79.90 ± 0.35	81.13 ± 0.26	80.08 ± 0.19	90.39 ± 0.90	75.34 ± 0.52	90.63 ± 0.33	56.67 ± 0.35	76.31 ± 0.20

The second major component of the complexity comes from constructing the similarity matrix for our loss function. For the loss, graph-based similarity via Personalized PageRank uses local partitioning with $O(2^b \log^3 m / \phi^2)$ time per pair, where ϕ is conductance and $b \in [1, \log m]$ is a constant, while tree-based co-occurrence via Jaccard requires $O(rn)$ preprocessing and $O(n^2)$ pairwise computations.

5 Experiments

In this section, we conduct experiments to evaluate the effectiveness of our proposed TopDSL framework by addressing the following research questions:

Q1: How does TopDSL perform compared with state-of-the-art methods for node-level and graph-level classification tasks?

Q2: How do different components of TopDSL (tree-guided augmentation and tree decomposition loss) contribute to the overall performance?

Q3: How sensitive is TopDSL to key hyperparameters such as base perturbation rate and similarity fusion weight?

5.1 Experimental Setup

Datasets. We conduct a comprehensive evaluation of TopDSL on 16 widely-used benchmark datasets, spanning both node-level and graph-level tasks to demonstrate its broad effectiveness. For graph classification, we select 8 datasets from the TUDataset benchmark [28]. These include four bioinformatics datasets: NCI1 and MUTAG, where graphs represent molecules to be classified by anti-cancer activity and mutagenicity, respectively; and PROTEINS and DD, where graphs are protein structures classified by function or as enzymes. We also use four social network datasets: IMDB-B, consisting of actor ego-networks classified by genre; COLLAB, a scientific

collaboration dataset; and RDT-B and RDT-M5K, which are datasets of Reddit discussion threads where graphs must be classified into communities or interaction patterns.

For node classification, we use 8 standard benchmarks. Three are citation networks: Cora, CiteSeer, and PubMed [23], where nodes are scientific papers connected by citations, and the task is to classify them by research area. We also use WikiCS, a network of computer science articles from Wikipedia. To evaluate performance on different structures, we include two co-purchase networks, Amazon-Computers and Amazon-Photo, where nodes are products and edges link co-purchased items. Finally, we use two large co-authorship networks, Coauthor-CS and Coauthor-Physics [40], where nodes are authors connected by co-authored publications.

Baselines. We compare TopDSL against a comprehensive and diverse set of baseline methods to ensure a thorough evaluation across both tasks. The baselines are categorized by their learning paradigm. For graph-level classification, we include four categories of methods. 1) *Supervised GNNs*: GCN [23] and the more powerful GIN [58], which serve as performance references. 2) *Graph Kernels*: Traditional methods that measure graph similarity, including Shortest Path (SP) [10], Graphlet (GK) [42], the influential Weisfeiler-Lehman subtree kernel (WL) [41], and Deep Graph Kernel (DGK) [59]. 3) *Unsupervised Methods*: Embedding techniques like NODE2VEC [19], SUB2VEC [2], and GRAPH2VEC [30] which learn representations without labels. 4) *Self-supervised Contrastive Methods*: A wide array of state-of-the-art models including the pioneering INFOGRAPH [44] and GRAPHCL [63]; methods with learnable augmentations like AD-GCL [45] and AUTOGCL [61]; and other recent strong performers such as JOAOV2 [62], RGCL [24], SIMGRACE [55], and SEGA [52].

For node-level classification, our comparison includes three categories. 1) *Supervised Methods*: A standard MLP that uses only node features, and GCN [23] as the canonical GNN baseline. 2) *Graph*

Table 2: Mean node classification accuracy for supervised and unsupervised models. The highest unsupervised performance is highlighted in bold, and the second-highest performance is underlined. OOM indicates the out-of-memory on a 24GB GPU.

Model	Cora	CiteSeer	PubMed	WikiCS	Amz-Comp.	Amz-Photo	Coauthor-CS	Coauthor-Phy.
MLP	47.92 $\pm$ 0.41	49.31 $\pm$ 0.26	69.14 $\pm$ 0.34	71.98 $\pm$ 0.42	73.81 $\pm$ 0.21	78.53 $\pm$ 0.32	90.37 $\pm$ 0.19	93.58 $\pm$ 0.41
GCN	81.54 $\pm$ 0.68	70.73 $\pm$ 0.65	79.16 $\pm$ 0.25	93.02 $\pm$ 0.11	86.51 $\pm$ 0.54	92.42 $\pm$ 0.22	93.03 $\pm$ 0.31	95.65 $\pm$ 0.16
DEEPWALK	70.72 $\pm$ 0.63	51.39 $\pm$ 0.41	73.27 $\pm$ 0.86	74.42 $\pm$ 0.13	85.68 $\pm$ 0.07	89.40 $\pm$ 0.11	84.61 $\pm$ 0.22	91.77 $\pm$ 0.15
NODE2VEC	71.08 $\pm$ 0.91	47.34 $\pm$ 0.84	66.23 $\pm$ 0.95	71.76 $\pm$ 0.14	84.41 $\pm$ 0.14	89.68 $\pm$ 0.19	85.16 $\pm$ 0.04	91.23 $\pm$ 0.07
VGAE	77.27 $\pm$ 0.86	67.46 $\pm$ 0.20	76.02 $\pm$ 0.52	75.55 $\pm$ 0.22	86.40 $\pm$ 0.30	92.13 $\pm$ 0.12	92.10 $\pm$ 0.31	94.43 $\pm$ 0.20
DGI	82.24 $\pm$ 0.63	71.82 $\pm$ 0.61	76.80 $\pm$ 0.30	75.42 $\pm$ 0.17	84.05 $\pm$ 0.42	91.62 $\pm$ 0.37	92.14 $\pm$ 0.55	94.54 $\pm$ 0.52
GMI	82.40 $\pm$ 0.57	71.74 $\pm$ 0.12	79.28 $\pm$ 0.94	74.79 $\pm$ 0.16	82.24 $\pm$ 0.39	90.81 $\pm$ 0.15	OOM	OOM
MVGRL	83.37 $\pm$ 0.65	<u>73.29 $\pm$ 0.36</u>	80.33 $\pm$ 0.61	77.55 $\pm$ 0.06	87.45 $\pm$ 0.17	91.77 $\pm$ 0.21	92.24 $\pm$ 0.31	95.30 $\pm$ 0.13
GRACE	81.88 $\pm$ 0.84	71.13 $\pm$ 0.42	80.88 $\pm$ 0.13	<u>79.37 $\pm$ 0.24</u>	86.48 $\pm$ 0.24	92.20 $\pm$ 0.16	92.90 $\pm$ 0.27	95.25 $\pm$ 0.26
GCA	82.41 $\pm$ 0.55	71.56 $\pm$ 0.19	80.73 $\pm$ 0.23	78.26 $\pm$ 0.39	87.92 $\pm$ 0.33	92.35 $\pm$ 0.53	92.65 $\pm$ 0.32	<u>95.52 $\pm$ 0.21</u>
BGRL	81.86 $\pm$ 0.32	72.10 $\pm$ 0.31	80.65 $\pm$ 0.42	79.28 $\pm$ 0.45	<u>89.21 $\pm$ 0.47</u>	92.28 $\pm$ 0.44	92.73 $\pm$ 0.41	95.31 $\pm$ 0.26
SUGRL	83.29 $\pm$ 0.38	73.11 $\pm$ 0.22	<u>81.96 $\pm$ 0.49</u>	78.88 $\pm$ 0.35	88.98 $\pm$ 0.20	92.87 $\pm$ 0.19	92.84 $\pm$ 0.24	94.80 $\pm$ 0.24
CCA-Ssg	<u>84.17 $\pm$ 0.44</u>	73.27 $\pm$ 0.30	81.91 $\pm$ 0.41	77.67 $\pm$ 0.29	88.88 $\pm$ 0.22	<u>93.14 $\pm$ 0.43</u>	<u>93.23 $\pm$ 0.16</u>	95.29 $\pm$ 0.11
TopDSL	84.72 $\pm$ 0.19	73.33 $\pm$ 0.25	83.81 $\pm$ 0.32	82.10 $\pm$ 0.29	90.77 $\pm$ 0.37	93.22 $\pm$ 0.11	93.59 $\pm$ 0.24	95.98 $\pm$ 0.13

Embedding Methods: Foundational random-walk based methods DEEPWALK [35] and NODE2VEC [19]. 3) *Graph Contrastive Learning Methods:* We include the seminal DGI [49] which contrasts local and global information; generative models like VGAE [22]; multi-view methods like MVGRL [20]; methods with adaptive augmentations like GCA [68]; non-contrastive approaches like BGRL [47]; and other state-of-the-art models like GRACE [67], GMI [34], CCA-Ssg [64], and SUGRL [27].

Protocol. We follow the standard evaluation protocols of previous state-of-the-art self-supervised graph learning methods. For node classification, we report the mean accuracy on the test set after 50 runs of training using pretrained node embeddings to train a linear classifier. The dataset is split into 10%/10%/80% for training, validation, and testing, respectively. For graph classification, we evaluate learned graph representations using a linear SVM classifier and report the mean 10-fold cross-validation accuracy across 5 runs. For each training fold, the linear SVM is tuned using cross-validation. The dataset is split into 80%/10%/10% for training, validation, and testing, respectively.

Implementation Details. We implement TopDSL using PyTorch Geometric on two NVIDIA A6000 GPUs. We adopt the Adam optimizer with learning rates tuned from {0.0001, 0.001, 0.01}, batch sizes from {16, 64, 128, 256, 512}, and embedding dimensions from {64, 128, 256, 512}. The base perturbation rate p_0 is tuned from {0.1, 0.2, 0.3, 0.4, 0.5}. We explore both combined and individual augmentation strategies. The similarity fusion weight α is searched over {0.01, 0.1, ..., 0.9, 0.99}.

5.2 Performance Comparison (Q1)

Performance on Graph-level Tasks. Table 1 summarizes the results for graph classification across 8 TUDataset benchmarks. TopDSL consistently achieves superior performance compared to all state-of-the-art baselines. For instance, on the PROTEINS dataset, TopDSL achieves an accuracy of 81.13%, significantly outperforming the next best unsupervised method, SEGA, by 5.13%. Similarly, on DD and MUTAG, TopDSL secures the top performance with scores of 80.08% and 90.39%, respectively, outperforming a wide

range of both kernel-based and contrastive learning baselines. On IMDB-B, TopDSL achieves 76.31%, which not only surpasses the strongest unsupervised baseline (SEGA, 73.60%) but also exceeds the powerful supervised method GIN by 2.61%. Across other dataset like COLLAB, our model demonstrates clear advantages, with performance gains of 1.24% over the second-best method. These consistent improvements demonstrate the effectiveness of our tree decomposition-guided augmentation strategy in preserving local structures.

Performance on Node-level Tasks. Table 2 presents the results for node classification on 8 benchmark datasets. The results show that TopDSL achieves state-of-the-art performance across all datasets. Notably, TopDSL not only outperforms the strongest unsupervised baselines but also consistently exceeds the performance of supervised methods like GCN. For example, on the PubMed dataset, TopDSL achieves an accuracy of 83.81%, surpassing the best-performing unsupervised method, SUGRL, by 1.85% and the supervised GCN baseline by a significant margin of 4.65%.

Similarly, on Cora and CiteSeer, TopDSL achieves state-of-the-art with accuracies of 84.72% and 73.33%, respectively. On larger datasets like Amazon-Computer and Coauthor-Physics, our model continues to lead, achieving improvements of 1.56% and 0.46% over the strongest competitors. These substantial gains validate our hypothesis that tree decomposition effectively captures multi-scale structural patterns. The dual similarity framework, which fuses graph-based proximity with tree-based co-occurrence patterns, enables the model to learn highly discriminative representations that respect both local topological relationships and the hierarchical contexts revealed by the decomposition.

5.3 Ablation Study (Q2)

To validate the individual contributions of our core components, we perform an ablation study by selectively deactivating the tree-guided adaptive augmentation (*Aug*) and the decomposition-aware structural loss (*Loss*). As detailed in Table 3, we test three variants: removing only the augmentation (*w/o Aug*), removing only the loss (*w/o Loss*), and removing both (*w/o Aug+Loss*). The results clearly

show that each component is vital to the model’s success. For instance, on the Cora dataset, removing the adaptive augmentation (w/o *Aug*) or the structural loss (w/o *Loss*) results in a significant performance drop of 2.61 and 2.08 percentage points, respectively. This consistent degradation across all tested datasets confirms that both the context-aware view generation and the structurally-informed objective are crucial for learning high-quality representations.

Furthermore, the performance degrades most severely when both modules are removed simultaneously, as seen in the *w/o Aug+Loss* setting. This variant, which resembles a standard contrastive learning baseline with random augmentations, performs significantly worse than the full TopDSL model. For example, on PubMed, the full model outperforms this baseline by 2.62 percentage points, and on Cora, the margin is even larger at 3.43 percentage points. This substantial gap highlights a strong synergistic effect between our two main contributions. The adaptive augmentation strategy produces semantically robust views by preserving critical structures, while the decomposition-aware loss provides the ideal objective to capitalize on these views. This synergy is a key factor in TopDSL achieving state-of-the-art performance.

Table 3: Ablation study on 4 datasets.

Method	Cora	CiteSeer	PubMed	WikiCS
w/o Aug+Loss	81.29 ± 0.28	70.82 ± 0.36	81.19 ± 0.34	79.79 ± 0.42
w/o Aug	82.11 ± 0.34	72.27 ± 0.22	82.32 ± 0.34	81.74 ± 0.31
w/o Loss	82.64 ± 0.26	71.70 ± 0.15	82.17 ± 0.29	80.88 ± 0.24
TopDSL	84.72 ± 0.19	73.33 ± 0.25	83.81 ± 0.32	82.10 ± 0.29

Table 4: Sensitivity analysis for perturbation rate p_0 .

p_0	Cora	CiteSeer	PubMed	WikiCS
0.1	84.70 ± 0.23	73.29 ± 0.25	83.81 ± 0.31	82.11 ± 0.26
0.2	84.62 ± 0.26	73.33 ± 0.16	83.62 ± 0.12	82.07 ± 0.29
0.3	84.29 ± 0.11	73.31 ± 0.29	82.89 ± 0.25	82.01 ± 0.25
0.4	83.36 ± 0.32	72.17 ± 0.14	82.91 ± 0.36	81.63 ± 0.31
0.5	82.28 ± 0.26	71.56 ± 0.23	82.35 ± 0.16	81.03 ± 0.49

5.4 Sensitivity Analysis (Q3)

We investigate the sensitivity of TopDSL to two hyperparameters: the base perturbation rate p_0 for our augmentation strategy and the similarity fusion weight α for our loss.

Base Perturbation Rate p_0 . As shown in Table 4, we analyze the impact of the base perturbation rate p_0 , which uniformly scales the magnitude of our adaptive augmentations. The results indicate that TopDSL demonstrates robust and superior performance at lower to moderate perturbation rates, typically within the range of $p_0 \in [0.1, 0.3]$. For instance, on the Cora and PubMed datasets, the peak performance is achieved at the lowest tested rate of $p_0 = 0.1$. On CiteSeer, the optimum is found at $p_0 = 0.2$. The model’s performance is relatively stable within this optimal range; for example, on WikiCS, the accuracy only varies by 0.1% as p_0 increases from 0.1 to 0.3. This desirable stability suggests the framework does not require fine-tuning this parameter for high performance.

Conversely, a downward trend in accuracy is observed across all datasets as the rate increases to 0.4 and beyond. On the Cora dataset, increasing p_0 from 0.1 to 0.5 leads to a performance drop of 2.42

Table 5: Parameter sensitivity analysis for weight α .

α	Cora	CiteSeer	PubMed	WikiCS
0.01	84.60 ± 0.25	73.33 ± 0.32	82.57 ± 0.27	81.94 ± 0.34
0.1	84.71 ± 0.21	73.16 ± 0.18	83.73 ± 0.32	82.08 ± 0.23
0.3	84.53 ± 0.15	73.30 ± 0.12	83.81 ± 0.24	82.04 ± 0.23
0.5	83.23 ± 0.18	72.83 ± 0.14	83.44 ± 0.18	82.10 ± 0.21
0.7	83.37 ± 0.20	72.39 ± 0.24	83.18 ± 0.32	81.71 ± 0.30
0.99	83.42 ± 0.32	72.12 ± 0.35	82.73 ± 0.43	81.57 ± 0.29

percentage points. This trend highlights a fundamental trade-off in contrastive learning: while a sufficient degree of perturbation is necessary to create diverse views that encourage learning invariant features, an excessively high rate can corrupt the graph’s essential semantic structure. By applying overly aggressive augmentations, even our adaptive policy risks removing too many critical nodes and edges, introducing more noise than signal and ultimately hindering the learning process. Our findings suggest that a rate between 0.1 and 0.3 strikes the right balance between generating meaningful augmentations and preserving structural integrity.

Similarity Fusion Weight α . Table 5 shows the model’s performance as we vary the weight α , which balances the influence of graph-based similarity (S_g , weighted by α) and tree-based similarity (S_t , weighted by $1-\alpha$). This parameter allows us to tailor the model’s objective to prioritize either local topological proximity or broader, hierarchically-informed structural roles. The results indicate that the optimal fusion is dataset-dependent, reflecting the unique structural properties of each graph. For instance, CiteSeer achieves its best result (73.33%) with a small $\alpha = 0.01$, suggesting a stronger reliance on the hierarchical information from tree decomposition. In contrast, WikiCS performs best at $\alpha = 0.5$ (82.10%), indicating a balanced need for both topological proximity and hierarchical context. Importantly, performance tends to degrade at the extremes (e.g., $\alpha = 0.99$), where one of the similarity signals is almost entirely ignored. This confirms that fusing both traditional graph topology and the novel tree-based structural context is important to learning graph representations.

6 Conclusion

In this paper, we proposed TopDSL, a novel self-supervised learning framework that integrates the principle of tree decomposition to capture the inherent hierarchical structure of graphs (e.g., computing power networks). Unlike methods that rely on global heuristics, TopDSL introduces a context-aware augmentation strategy guided by importance scores computed within local decomposition bags, preserving critical structures. Furthermore, we designed a new contrastive loss that fuses traditional graph proximity with a novel, tree-based similarity metric. This dual approach enables the learning of robust representations sensitive to multi-scale structural roles. Extensive experiments on various benchmarks demonstrate that TopDSL achieves SOTA performance, confirming the value of integrating topological priors into graph representation learning.

Acknowledgments

This research is supported by the National Natural Science Foundation of China under Grant No. 62432003.

References

- [1] Aaron B Adcock, Blair D Sullivan, and Michael W Mahoney. 2016. Tree decompositions and social graphs. *Internet Mathematics* 12, 5 (2016), 315–361.
- [2] Bijaya Adhikari, Yao Zhang, Naren Ramakrishnan, and B Aditya Prakash. 2018. Sub2vec: Feature learning for subgraphs. In *Pacific-Asia Conference on KDDM*. Springer, 170–182.
- [3] Eyal Amir. 2010. Approximation algorithms for treewidth. *Algorithmica* 56, 4 (2010), 448–479.
- [4] Reid Andersen, Fan Chung, and Kevin Lang. 2006. Local graph partitioning using pagerank vectors. In *IEEE Symposium on Foundations of Computer Science*. IEEE, 475–486.
- [5] Stefan Arnborg, Derek G Corneil, and Andrzej Proskurowski. 1987. Complexity of finding embeddings in a k-Tree. *SIAM Journal on Algebraic Discrete Methods* 8, 2 (1987), 277–284.
- [6] Guillaume Blin, Florian Sikora, and Stephane Vialette. 2010. Querying graphs in protein-protein interactions networks using feedback vertex set. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 7, 4 (2010), 628–635.
- [7] Hans L Bodlaender. 1994. A tourist guide through treewidth. *Developments in Theoretical Computer Science* 6 (1994), 1.
- [8] Hans L Bodlaender. 1998. A partial k-arboretum of graphs with bounded treewidth. *Theoretical Computer Science* 209, 1-2 (1998), 1–45.
- [9] Hans L Bodlaender and Arie MCA Koster. 2010. Treewidth computations I. Upper bounds. *Information and Computation* 208, 3 (2010), 259–275.
- [10] Karsten M Borgwardt and Hans-Peter Kriegel. 2005. Shortest-path kernels on graphs. In *International Conference on Data Mining*. IEEE, 8 pp.–.
- [11] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning*. PMLR, 1597–1607.
- [12] Marek Cygan, Fedor V Fomin, Łukasz Kowalik, Daniel Lokshtanov, Dániel Marx, Marcin Pilipczuk, Michał Pilipczuk, and Saket Saurabh. 2015. *Parameterized algorithms*. Vol. 5. Springer.
- [13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.
- [14] Banu Dost, Tomer Shlomi, Nitin Gupta, Eytan Ruppin, Vineet Bafna, and Roded Sharan. 2008. QNet: a tool for querying protein interaction networks. *Journal of Computational Biology* 15, 7 (2008), 913–925.
- [15] Tim S Evans and Renaud Lambiotte. 2009. Line graphs, link partitions, and overlapping communities. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 80, 1 (2009), 016105.
- [16] Yin Fang, Qiang Zhang, Ningyu Zhang, Zhuo Chen, Xiang Zhuang, Xin Shao, Xiaohui Fan, and HuaJun Chen. 2023. Knowledge graph-enhanced molecular contrastive learning with functional prompt. *Nature Machine Intelligence* 5, 5 (2023), 542–553.
- [17] Steve Gregory. 2009. Finding overlapping communities using disjoint community detection algorithms. In *Complex networks*. Springer, 47–61.
- [18] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. 2020. Bootstrap your own latent—a new approach to self-supervised learning. *Advances in neural information processing systems* 33 (2020), 21271–21284.
- [19] Aditya Grover and Jure Leskovec. 2016. Node2vec: Scalable feature learning for networks. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 855–864.
- [20] Kaveh Hassani and Amir Hosein Khasahmadi. 2020. Contrastive multi-view representation learning on graphs. In *Proceedings of the International Conference on Machine Learning*. PMLR, 4116–4126.
- [21] Wei Jin, Tyler Derr, Haochen Liu, Yiqi Wang, Suhang Wang, Zitao Liu, and Jiliang Tang. 2020. Self-supervised learning on graphs: Deep insights and new direction. *arXiv preprint arXiv:2006.10141* (2020).
- [22] Thomas Kipf and Max Welling. 2016. Variational graph auto-encoders. *Bayesian Deep Learning, NIPS 2016 Workshop* (2016).
- [23] Thomas N Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- [24] Sihang Li, Xiang Wang, An Zhang, Yingxin Wu, Xiangnan He, and Tat-Seng Chua. 2022. Let invariant rationale discovery inspire graph contrastive learning. In *Proceedings of the International Conference on Machine Learning*. 13052–13065.
- [25] Yixin Liu, Ming Jin, Shirui Pan, Chuan Zhou, Yu Zheng, Feng Xia, and Philip S Yu. 2022. Graph self-supervised learning: A survey. *IEEE Transactions on Knowledge and Data Engineering* 35, 6 (2022), 5879–5900.
- [26] Silviu Maniu, Pierre Senellart, and Suraj Jog. 2019. An experimental study of the treewidth of real-world graph data (extended version). *arXiv preprint arXiv:1901.06862* (2019).
- [27] Yujie Mo, Liang Peng, Jie Xu, Xiaoshuang Shi, and Xiaofeng Zhu. 2022. Simple unsupervised graph representation learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 7797–7805.
- [28] Christopher Morris, Nils Morten Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. TUDataset: A collection of benchmark datasets for learning with graphs. In *ICML 2020 Workshop on Graph Representation Learning and Beyond*.
- [29] Allan H Murphy. 1996. The Finley affair: A signal event in the history of forecast verification. *Weather and Forecasting* 11, 1 (1996), 3–20.
- [30] Annamalai Narayanan, Mahinthan Chandramohan, Rajasekar Venkatesan, Lihui Chen, Yang Liu, and Shantanu Jaiswal. 2017. Graph2vec: Learning distributed representations of graphs. *arXiv:1707.05005* (2017).
- [31] Mark EJ Newman, Steven H Strogatz, and Duncan J Watts. 2001. Random graphs with arbitrary degree distributions and their applications. *Physical review E* 64, 2 (2001), 026118.
- [32] Rolf Niedermeier. 2006. *Invitation to fixed-parameter algorithms*. Vol. 31. OUP Oxford.
- [33] Milena Oehlers and Benjamin Fabian. 2021. Graph metrics for network robustness—a survey. *Mathematics* 9, 8 (2021), 895.
- [34] Zhen Peng, Wenbing Huang, Minnan Luo, Qinghua Zheng, Yu Rong, Tingyang Xu, and Junzhou Huang. 2020. Graph representation learning via graphical mutual information maximization. In *The Web Conference*. 259–270.
- [35] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 701–710.
- [36] Neil Robertson and Paul D Seymour. 1986. Graph minors. II. Algorithmic aspects of tree-width. *Journal of Algorithms* 7, 3 (1986), 309–322.
- [37] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. 2019. Droppedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903* (2019).
- [38] Madeline C Schiappa, Yogesh S Rawat, and Mubarak Shah. 2023. Self-supervised learning for videos: A survey. *Comput. Surveys* 55, 13s (2023), 1–37.
- [39] Kartik Sharma, Yeon-Chang Lee, Sivagami Nambi, Aditya Salian, Shlok Shah, Sang-Wook Kim, and Srijan Kumar. 2024. A survey of graph neural networks for social recommender systems. *Comput. Surveys* 56, 10 (2024), 1–34.
- [40] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of Graph Neural Network Evaluation. *Relational Representation Learning Workshop, NeurIPS* (2018).
- [41] Nino Shervashidze, Pascal Schweitzer, Erik Jan Van Leeuwen, Kurt Mehlhorn, and Karsten M Borgwardt. 2011. Weisfeiler-Lehman graph kernels. *Journal of Machine Learning Research* 12, 9 (2011).
- [42] Nino Shervashidze, SVN Vishwanathan, Tobias Petri, Kurt Mehlhorn, and Karsten Borgwardt. 2009. Efficient graphlet kernels for large graph comparison. In *Artificial Intelligence and Statistics*. PMLR, 488–495.
- [43] Hamed Shirzad, Hossein Hajimirsadeghi, Amir H Abdi, and Greg Mori. 2021. TD-GEN: Graph Generation With Tree Decomposition. *arXiv preprint arXiv:2106.10656* (2021).
- [44] Fan-Yun Sun, Jordon Hoffman, Vikas Verma, and Jian Tang. 2020. InfoGraph: Unsupervised and Semi-supervised Graph-Level Representation Learning via Mutual Information Maximization. In *International Conference on Learning Representations*.
- [45] Sushel Suresh, Pan Li, Cong Hao, and Jennifer Neville. 2021. Adversarial graph augmentation to improve graph contrastive learning. *Advances in Neural Information Processing Systems* 34 (2021), 15920–15933.
- [46] Shiyin Tan, Dongyuan Li, Renhe Jiang, Ying Zhang, and Manabu Okumura. 2024. Community-invariant graph contrastive learning. *arXiv preprint arXiv:2405.01350* (2024).
- [47] Shantanu Thakoor, Corentin Tallec, Mohammad Gheshlaghi Azar, Mehdi Azabou, Eva L Dyer, Remi Munos, Petar Veličković, and Michal Valko. 2022. Large-Scale Representation Learning on Graphs via Bootstrapping. In *International Conference on Learning Representations*.
- [48] Thomas W Valente and Kayo Fujimoto. 2010. Bridging: locating critical connectors in a network. *Social Networks* 32, 3 (2010), 212–220.
- [49] Petar Velickovic, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2019. Deep Graph Infomax. In *International Conference on Learning Representations*.
- [50] Yejiang Wang, Yuhai Zhao, Daniel Zhengkui Wang, and Ling Li. 2023. GALOPA: Graph Transport Learning with Optimal Plan Alignment. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. 9117–9130.
- [51] Yejiang Wang, Yuhai Zhao, Zhengkui Wang, Ling Li, Jiapu Wang, Fangting Li, Miaomiao Huang, Shirui Pan, and Xingwei Wang. 2025. Equivalence is All: A Unified View for Self-supervised Graph Learning. In *Forty-second International Conference on Machine Learning*. <https://openreview.net/forum?id=ZAlII9wL5i>
- [52] Junran Wu, Xueyan Chen, Bowen Shi, Shangzhe Li, and Ke Xu. 2023. SEGA: Structural Entropy Guided Anchor View for Graph Contrastive Learning. In *Proceedings of the International Conference on Machine Learning*. Vol. 202. 37293–37312.
- [53] Lingfei Wu, Peng Cui, Jian Pei, Liang Zhao, and Le Song. 2022. Graph Neural Networks. In *Graph Neural Networks: Foundations, Frontiers, and Applications*.

- Springer, 27–37.
- [54] Lirong Wu, Haitao Lin, Cheng Tan, Zhangyang Gao, and Stan Z Li. 2021. Self-supervised learning on graphs: Contrastive, generative, or predictive. *IEEE Transactions on Knowledge and Data Engineering* 35, 4 (2021), 4216–4235.
 - [55] Jun Xia, Lirong Wu, Jintao Chen, Bozhen Hu, and Stan Z Li. 2022. SimGRACE: A Simple Framework for Graph Contrastive Learning without Data Augmentation. In *Proceedings of the ACM Web Conference 2022*. 1070–1079.
 - [56] Dongkuan Xu, Wei Cheng, Dongsheng Luo, Haifeng Chen, and Xiang Zhang. 2021. Infogcl: Information-aware graph contrastive learning. *Advances in Neural Information Processing Systems* 34 (2021), 30414–30425.
 - [57] Jinbo Xu, Feng Jiao, and Bonnie Berger. 2005. A tree-decomposition approach to protein structure prediction. In *2005 IEEE Computational Systems Bioinformatics Conference*. IEEE, 247–256.
 - [58] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*.
 - [59] Pinar Yanardag and SVN Vishwanathan. 2015. Deep graph kernels. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1365–1374.
 - [60] Hua-Ting Yao, Bertrand Marchand, Sarah J Berkemer, Yann Ponty, and Sebastian Will. 2024. Infrared: a declarative tree decomposition-powered framework for bioinformatics. *Algorithms for Molecular Biology* 19, 1 (2024), 13.
 - [61] Yihang Yin, Qingzhong Wang, Siyu Huang, Haoyi Xiong, and Xiang Zhang. 2022. Autogcl: Automated graph contrastive learning via learnable view generators. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 8892–8900.
 - [62] Yuning You, Tianlong Chen, Yang Shen, and Zhangyang Wang. 2021. Graph Contrastive Learning Automated. In *Proceedings of the International Conference on Machine Learning*. 12121–12132.
 - [63] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph Contrastive Learning with Augmentations. *Advances in Neural Information Processing Systems* 33 (2020), 5812–5823.
 - [64] Hengrui Zhang, Qitian Wu, Junchi Yan, David Wipf, and Philip S Yu. 2021. From canonical correlation analysis to self-supervised graph neural networks. *Advances in Neural Information Processing Systems* 34 (2021), 76–89.
 - [65] Shichang Zhang, Ziniu Hu, Arjun Subramonian, and Yizhou Sun. 2020. Motif-driven contrastive learning of graph representations. *arXiv preprint arXiv:2012.12533* (2020).
 - [66] Yuhai Zhao, Yejiang Wang, Zhengkui Wang, Wen Shan, Miaomiao Huang, and Xingwei Wang. 2025. Graph Contrastive Learning with Progressive Augmentations. In *Proceedings of ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2079–2088.
 - [67] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2020. Deep graph contrastive representation learning. *arXiv:2006.04131* (2020).
 - [68] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2021. Graph contrastive learning with adaptive augmentation. In *The Web Conference*. 2069–2080.